Personal Study Guide & Interview Prep

Self-Reference Manual

1 How Do I Explain This Project in 60 Seconds?

"I built a production-grade multi-agent LLM system designed to tackle complex, multi-step queries that standard conversational AI fails at. Instead of a single monolithic prompt, I implemented a decentralized architecture where seven specialized agents—like Orchestrator, Retrieval, and Critique—collaborate around a central Shared Context. It's technically interesting because I built a strict context budget manager using tiktoken to prevent runaway costs, and a Self-Improving Loop where a Meta-Agent automatically proposes prompt rewrites based on structured evaluation metrics. I primarily used DeepSeek due to its excellent coding and reasoning performance, but it's entirely model-agnostic. The most impressive detail is the human-in-the-loop rewrite approval system, which creates an observable, continuously improving architecture."

2 How Does the Orchestrator Actually Work?

When a query arrives, it is registered in the `SharedContext` and picked up by the Celery worker. The first function called is `run_orchestrator`. The orchestrator sends the current pipeline state (like completed agents and remaining budget) and the query to DeepSeek. It requires a strict JSON response containing `reasoning`, `next_agents`, and `context_budget_allocation`. If the LLM hallucinates malformed JSON or times out, the code explicitly catches the `JSONDecodeError` and gracefully falls back to a sequential `["decomposition", "retrieval", "critique", "synthesis"]` pipeline to guarantee execution.

3 How Does the Budget Manager Work and Why Does It Matter?

The budget manager acts like a firewall against infinite loops and massive token bills. I used `tiktoken` with the `cl100k_base` encoding (standard for modern models) to measure string lengths accurately. The `AGENT_BUDGETS` dictionary grants the Orchestrator 4000 tokens and Synthesis 6000. When an agent runs, it calls `consume_budget`. If it tries to use more tokens than remain, the function logs a `budget_violation` to the PostgreSQL `execution_logs` table and returns `False`. The agent detects this and aborts early. This is critical in production because it turns silent, expensive failures into explicit, manageable logs.

4 How Does Multi-Hop Retrieval Work?

Retrieval executes in stages. Hop 1 performs a broad search based on the decomposition tasks. The agent evaluates the returned data to determine what context is still missing. It then crafts a targeted Hop 2 query. The retrieved texts are saved into the context as `chunk_1` and `chunk_2`. If a

tool returns `TOOL_TIMEOUT`, the orchestrator allows up to two retries, incrementing the `retry_count` inside the `ToolCall` schema for full auditability.

5 How Does the Critique Agent Avoid Flagging the Whole Output?

I designed the Critique agent to evaluate outputs claim-by-claim. Instead of giving a generic "7/10" to an entire essay, it outputs an array of `ClaimScore` objects. Each object extracts a specific `claim_text`, assigns a `confidence` float, and sets a boolean `flagged` field with a `flag_reason`. This granular approach allows the Synthesis agent to surgically rewrite only the contradictory parts rather than generating an entirely new draft from scratch.

6 How Does the Self-Improving Loop Work End to End?

Imagine the retrieval agent hallucinates during an adversarial test case. The eval harness runs and scores "Citation Accuracy" terribly (e.g., 0.1). This run is saved. The Meta-Agent wakes up, queries the DB, and spots this failure. It maps the poor citation score to the Retrieval agent, and asks DeepSeek: "Look at this failure. Propose a new system prompt." DeepSeek generates a rewrite, which is saved as `pending`. I manually hit the `/rewrites/{id}` endpoint to approve it. The system updates the `AgentPrompt` table. The next time the Retrieval agent runs, `get_active_prompt()` pulls this new prompt dynamically. Finally, the system reruns the test case, logs the new score, and calculates the `PerformanceDelta`.

7 What Are the Failure Contracts for Each Tool?

- **web_search**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_MALFORMED`
- **code_executor**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_MALFORMED`, `TOOL_ERROR`
- **data_lookup**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_SQL_ERROR`

8 How Does SSE Streaming Work in This System?

Server-Sent Events (SSE) allow the server to push updates to the client over a single HTTP connection. The system emits events like `agent_start`, `tool_call_start`, `context_budget_update`, and `pipeline_complete` as the Celery pipeline runs. Currently, it emits event-level updates (entire chunks of progress). To achieve true token-level streaming, I would need to pass an async generator through the LLM client call itself, allowing the UI to render text as the LLM thinks.

9 What Would I Build Next and Why?

1. **Secure Code Sandbox**: Integrating Firecracker microVMs. Python `eval` is dangerous; a secure sandbox is mandatory for production.
2. **True Token Streaming**: Modifying the SSE pipeline to yield partial tokens. This is crucial for perceived UX latency.
3. **Auth Middleware**: Adding JWT verification. The API is currently open, which is unacceptable for enterprise deployment.

10 What Are the Hardest Technical Questions I Might Get Asked?

- **Why SharedContext instead of direct calls?** Direct calls create tight coupling and infinite loops. A SharedContext acts like a Blackboard, making state fully observable and strictly controlling token budgets.
- **Concurrent requests?** The Celery worker handles concurrency effortlessly, writing states back to a Postgres database mapped by `job_id`.
- **Lossy vs Lossless compression?** The compression agent uses an LLM to summarize older context (lossy) to fit within token limits, rather than arbitrary truncation.

11 Honest Weaknesses I Should Acknowledge

- The Search tool is currently a stub. I should have integrated the Tavily API, but I prioritized architectural completeness.
- The logging is synchronous in Postgres, which could block high-throughput API calls. I should move logs to an asynchronous Kafka queue.

12 Quick Reference Cheat Sheet

Agents: Orchestrator (4k), Decomposition (3k), Retrieval (5k), Critique (4k), Synthesis (6k), Compression (3k), Meta (4k).
Endpoints: `POST /query`, `GET /jobs/{id}/trace`, `GET /evals/latest`, `POST /rewrites/{id}`, `POST /evals/rerun`.
Tables: `jobs`, `eval_runs`, `prompt_rewrites`, `performance_deltas`, `agent_prompts`, `execution_logs`.